2AMM10 Assignment 2: Autonomous Driving

May 28, 2026

1 Introduction

Hannah Wolffson, a world-renowned Formula 1 chief engineer known for her expertise in high-performance systems and data-driven engineering, has been approached by a leading German automotive manufacturer to guide the transition toward next-generation autonomous vehicles. While her background is rooted in motorsport, she has become a strong advocate for integrating modern AI methods into safety-critical systems.

Recognizing that these challenges are not only technological but also conceptual, she launches an initiative to recruit Europe's most talented MSc students (that would be you :). Your objective is to assist her in identifying suitable learning architectures capable of capturing the visual and geometric complexities encountered in autonomous driving systems.

2 Assignment Description

Due to budget restrictions and fierce market competition, Professor Wolffson is eager for efficient and rapid prototyping. You are therefore given the following **preprocessed** datasets, each corresponding to a different perception task:

- *DrowsyFaceGraphDataset*: A graph dataset associated with driver drowsiness detection. It consists of 400 training and 100 test upper-face mesh graphs. Each graph corresponds to a driver who is either *awake* or *drowsy*.
- *BDDObjectClassificationDataset*: A dataset associated with **multi-class object classification**. It consists of cropped object regions extracted from BDD100K[1]. Each sample contains a single object belonging to one of eight classes. For each class, 300 training samples and 30 test samples are provided.

Fig. 1 illustrates samples from the datasets used throughout this assignment.

[1]BDD100K is a large-scale driving dataset containing annotations for multiple autonomous-driving perception tasks, including object detection, lane marking, drivable-area segmentation, and scene understanding.

Figure 1: Examples from the provided datasets. Top: samples from the *DrowsyFaceGraphDataset*, where driver upper-faces are represented as facial landmark graphs. Bottom: samples from the *BDDObjectClassificationDataset*.

2.1 Task 1: Driver Drowsiness Detection

An important component of autonomous driving is detecting when a driver is becoming drowsy or about to fall asleep. However, relying solely on standard RGB cameras can be problematic, particularly under low-light or nighttime driving conditions, where poor illumination may significantly reduce image quality and reliability.

Our customer uses a sophisticated driver-monitoring system that combines infrared and depth sensors with geometric facial-tracking software to construct upper-face mesh graphs regardless of lighting conditions.

Your objective is to design and train a neural-network architecture, which leverages the dataset **symmetries**, capable of detecting whether a driver is awake or drowsy. The model must be implemented **from scratch**. In safety-critical applications, selecting appropriate evaluation metrics is equally, if not more, important than selecting the architecture itself. In this setting, failing to detect a drowsy driver may prevent the vehicle from triggering an alarm or safety intervention, potentially leading to an accident.

You are asked to:

1. design and train a suitable neural-network architecture for this task.
2. log and visualize the training and test losses throughout training.
3. compute:
   - a metric of the fraction of truly drowsy drivers that are predicted as drowsy by the model,
   - and a metric of the fraction of drivers predicted as drowsy by the model that are truly drowsy.
4. evaluate your trained model on the provided test set and visualize the resulting metrics in a single figure.

Baseline Requirement: As a baseline requirement, your model must achieve a score of at least **0.6** on **both** evaluation metrics on the **test** set.

2.2 Task 2: Object Classification on Faulty Sensors

Our customer has informed us that a massive batch of their visual processing chips is defective. Replacing these units would lead to bankruptcy, so we must find a software-level solution for their hardware pipeline.

A race condition exists between the camera's internal clock and the data bus. Instead of writing the image to memory in a clean row-by-row sequence, the system writes the image as shuffled **8x8** patches. While each individual patch is captured correctly, their spatial locations become corrupted. Every captured frame therefore resembles a digital jigsaw puzzle, where the pieces are correct but their arrangement is random. An example is shown in Fig. 2.

Figure 2: (Left) Original image. (Right) Image corrupted by shuffled 8x8 patches.

Your objective is to design and train, **from scratch**, a neural-network architecture for multi-class object classification whose predictions are **invariant to the sensor corruption process**. As a restriction, you are **not allowed** to apply global pooling operations[2] (i.e global max/avg pooling) to the patch embeddings.

To simulate the corruption process during evaluation, you can use the image transformation class `PatchShuffle` on the test dataloader of the `BDDObjectClassificationDataset`.

You are asked to:

1. design and train a suitable neural-network architecture for this task using uncorrupted training images, and evaluate it on corrupted test images.
2. compute the same two evaluation metrics introduced in Task 1, for each object class individually.
3. visualize the per-class metrics in a single figure.
4. aggregate the per-class metrics across all classes on the corrupted test set. Then re-evaluate the model on the clean test set without the `PatchShuffle` corruption transformation and report the resulting **aggregated** metrics for both test sets (4 scalar values).

Baseline Requirement: As a baseline requirement, your model must achieve a score of at least **0.3** on **both aggregated** evaluation metrics on the **noisy test** set.

[2]Global pooling maps a tensor $x \in \mathbb{R}^{c \times h \times w}$ to a vector in $\mathbb{R}^c$ by aggregating over the spatial dimensions h and w.

2.3 Task 3: Object Classification with Transfer Learning

In this final task, you are asked to evaluate whether a general-purpose pretrained vision model can produce useful feature representations for object classification on the object classification dataset *BDDObjectClassificationDataset*.

Unlike Task 2, where robustness to the *PatchShuffle* sensor corruption process was explicitly investigated, this task considers a future hardware revision in which the sensor corruption issue has been resolved. Consequently, all experiments in this task must be performed using the standard **uncorrupted** version of the dataset.

You must employ a pretrained model of your choice belonging to the same architecture family[3] as the model developed in Task 2. The pretrained encoder must be used as a **frozen feature extractor**, meaning that its pretrained parameters are kept fixed and are not updated during training.

You are asked to:

1. extract feature representations for all training and test images using the frozen pretrained encoder.
2. perform **linear probing** by training[4] a learnable linear classifier on top of the frozen pretrained model representations.
3. perform **k-nearest neighbor (kNN) classification** directly in the pretrained model representations using a **single** value of k.
4. compare the performance of:
   - the pretrained model with linear probing,
   - the pretrained model with kNN classification,
   - and the model trained from scratch in Task 2

   using the previously introduced **aggregated** test metrics,
5. visualize the resulting comparison in a single figure.

3 Deliverables

The submission of this assignment is done in ANS. There are two deliverables:

- Implementation of the solution.
- Description and explanation of your approach, formatted as answers to the questions in ANS.

For the implementation, a skeleton Jupyter Notebook with functions that load the provided data for each task is provided. Please submit your code in the provided skeleton Jupyter Notebook.

You need to upload:

[3]For example, convolutional neural networks (ConvNets), Vision Transformers (ViTs), Graph Neural Networks (GNNs), and recurrent neural networks (RNNs) each constitute distinct architecture families. Multiple pretrained variants typically exist within each family (e.g., ResNet18, ResNet34, VGG13, VGG16).

[4]Since pretrained models can be computationally expensive, the linear classifier does not need to be trained for more than 10 epochs.

- the Jupyter Notebook (`.ipynb`),
- and a PDF version (`.pdf`) of the notebook with the execution output.

The results used to answer each question should be present in the notebook output. Make sure that the important components of the answers are visible, match the reported results, and are reproducible.

Please generate the PDF after running all cells of your solution. The maximum upload size is 25MB; you may need to remove some large image outputs before uploading.

4 Use of AI Tools

LLMs may be used for assistance with code implementation, debugging, and understanding programming errors. However, they may not be used to directly generate the final written answers submitted to ANS.

If you use an LLM for coding assistance, you must provide:

- the model(s) used,
- and the prompts used,

in the corresponding ANS section.

Remark: You are **strongly encouraged** to read the guidelines on the use of AI found on the Canvas course page: Guidelines on the use of AI for completing the assignments in 2AMM10 Deep Learning 2025.pdf

5 Important Notes

Unless explicitly stated otherwise, models should be trained from scratch.

The only exception is Task 3, where the use of a pretrained frozen encoder is required. In Task 3, only the linear probing classifier may be trained; the pretrained encoder weights must remain frozen.